

# Deep RL

## I Assignment 1

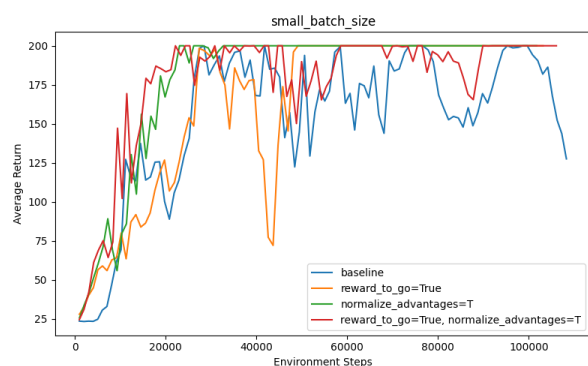
### I.1 Flow Matching Policy vs. MSE Policy

The flow policy agent takes actions to achieve the goal in a more orderly and consistent way. In contrast the MSE policy agent either achieves the goal quickly and efficiently or it gets very off track and performs actions that appear not to be leading directly toward pushing the T to the target location. A few hypotheses as to what might be going wrong here:

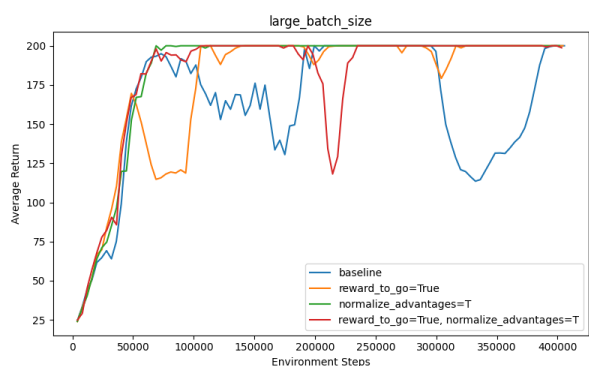
- the MSE policy isn't as equipped to handle multimodal distributions and when in a scenario where it should pick a mode it instead splits the difference leading to a suboptimal action.
- the MSE policy may be less consistent because its predictions are worse than the flow policy and as a result it gets out of distribution more often.

## II Assignment 2

### II.1 3.2 Questions



(a) Small Batch Size

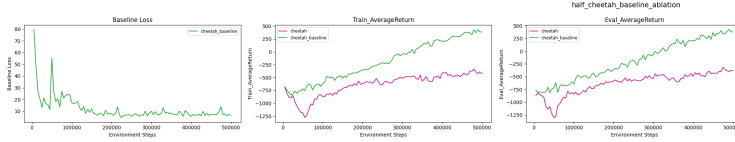


(b) Large Batch Size

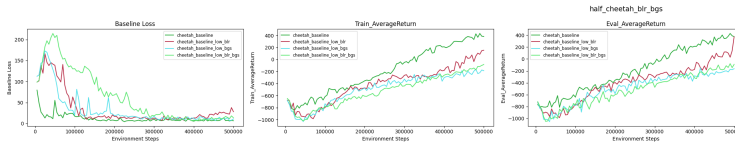
- Which value estimator has better performance without advantage normalization: the trajectory-centric one, or the one using reward-to-go?
  - Reward to go (RTG) has better performance without advantage normalization. You can see that RTG converges to 200 in both the large and small batch cases whereas the baseline does not in the small batch case.
- Between the two value estimators, why do you think one is generally preferred over the other?
  - The RTG estimator delivers higher avg return and lower stdev return when advantage normalization is not used.

- Did advantage normalization help?
  - Yes, this was the most significant impactful change. Turning advantage normalization on increased the speed of learning and decreased the variance.
- Did the batch size make an impact?
  - Yes, the returns are lower stdev for larger batch size.

## II.2 4.2 Questions



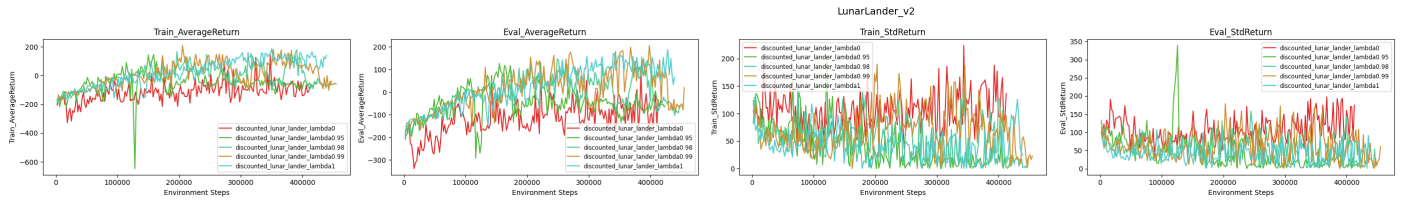
(a) Half Cheetah: Baseline Ablation



(b) Half Cheetah: Decreased Baseline Learning Rate / Gradient Steps

- Run another experiment with a decreased number of baseline gradient steps (-bgs) or baseline learning rate (-blr). How does this affect (a) the baseline learning curve and (b) the performance of the policy
  - (a) reducing the baseline LR or gradient steps increases the amount of env steps required for the baseline loss to plateau at a low value. In addition, changing these values causes the loss to initially increase vs. decrease. None of these variants reach comparable loss as the default hyper-param run, and the best loss of any variant is 68% higher. (b) all variants lag on avg. train return while for avg. eval return all variants lag except for low LR, which catches up in the final iters.

## II.3 5 Questions



(a) LunarLander-v2 Learning Curves

- Describe in words how  $\lambda$  affected task performance.
  - We see that increasing  $\lambda$  improves the avg return. In particular, it improves the avg return for the second half of training.
- Consider the parameter  $\lambda$ . What does  $\lambda = 0$  correspond to? What about  $\lambda = 1$ ? Relate this to the task performance in LunarLander-v2 in one or two sentences.

- $\lambda = 0$  corresponds to  $A_{\mathbf{GAE}}^\pi = \hat{A}^\pi(s_t, a_t) = r(s_t, a_t) + \gamma V(s_{t+1}) - V(s_t)$
- whereas  $\lambda = 1$  corresponds to equal weight being placed on each  $A_n^\pi(s_t, a_t)$
- $\lambda = 0$  is high bias low variance, while  $\lambda = 1$  is low bias high variance
- In the graphs we see  $\lambda = 0$  has the worst performance while  $\lambda = 1$  has the best performance. One hypothesis for why this may be the case is that with  $\lambda = 0$  there is too much bias i.e.  $V_\phi^\pi$  is not a reliable estimate of the value function, which causes underperformance. Therefore, we benefit from  $\lambda = 1$ , which gives us lower bias at the cost of higher variance.